

Hardware handover — running this on Nighthawk (or any backend we can't reach)

What we need from you: one shadow-Hadamard data point on better hardware than we have access to. Our account is on the IBM open plan and can only reach `ibm_fez`, `ibm_marrakesh` and `ibm_kingston` (all 156-qubit Heron-class).

UPDATE 2026-08-14 — the Part A ask below is DONE. A teammate ran it on `ibm_miami`, which is a **Nighthawk-class device**: its dump records 436 directed coupling edges = 218 couplers, exactly Nighthawk's published square-lattice spec (120 qubits), vs 176 couplers on the heavy-hex Herons. Results in `real_machine/`, analysed as R031/R053. **Still open: §2b (Track B) and §2c (AQC) have not run on Nighthawk** — those are now the asks, and §2c gains extra interest on a square lattice, where our routed 2D advantage (R052) was measured only against heavy-hex.

Budget: **2 circuits × 2000 shots ≈ 3 seconds of QPU time.** That is the whole ask.

0. What to copy (the whole handover is 10 files)

Verified by copying exactly these into an empty directory and running both dry-runs there; both reproduced the numbers below identically. Everything resolves its own paths from `__file__`, so any location works. **Nothing else in the repo is needed.**

file	why
<code>hardware_run.py</code>	provides <code>load_notebook_definitions()</code> / <code>get_model()</code> ; also the Part A tool
<code>shadow_hadamard_challenge_PARTICIPANT.ipynb</code>	the definitions are exec'd out of it — no kernel, never executed
<code>evidence/scripts/track_b_hardware_prep.py</code>	§2b go/no-go sizing, no QPU time
<code>evidence/scripts/track_b_hardware_submit.py</code>	§2b validate-then-submit
<code>evidence/scripts/track_b_hardware_fetch.py</code>	§2b analysis
<code>evidence/scripts/aqc_hardware_submit.py</code>	§2c validate-then-submit
<code>evidence/scripts/aqc_hardware_fetch.py</code>	§2c analysis
<code>evidence/scripts/track_b_baselines.py</code>	<i>optional</i> — echo/Hilbert-Schmidt costs, classical only
<code>CONVENTIONS.md</code>	<i>optional</i> — read §5 if any sign looks wrong
<code>HANDOVER.md</code>	this file

Or simply `git clone` the repo, which is public: <https://github.com/mnsh0409/Qiskit-Hackathon-2026>

Pick one job if you only have time for one: §2c (AQC). It is the shortest (72,000 shots), and it is the only one of ours whose *conclusion* improves with a better device rather than just its error bars.

Both submit scripts refuse to run if their statevector pre-flight fails. Leave that in — it is what caught an endianness bug and a phase bug before either reached a QPU.

Smoke test before you submit anything

```
python evidence/scripts/track_b_hardware_submit.py --dry-run # expect 116 circuits, arms A-D
python evidence/scripts/aqc_hardware_submit.py --dry-run # expect pre-flight PASSED
```

Expected output from the second, verified on a clean copy:

```
worst |chi_circuit - chi_exact|: exact 4.00e-16, trotter 1.47e-02, AQC+phase 7.90e-03  
PRE-FLIGHT PASSED
```

1. Setup (~5 minutes)

```
git clone <this repo> && cd 2026  
python -m venv .venv && source .venv/bin/activate  
pip install "qiskit==2.5.1" "qiskit-aer==0.17.2" "numpy<2.5" "scipy<1.18" \  
matplotlib pylatexenc qiskit-ibm-runtime
```

Two environment gotchas, both real:

- requirements.txt pins numpy==2.5.2 / scipy==1.18.0, which **require Python 3.12**. On 3.11 those resolve to nothing and the install fails. We ran on Python 3.11 with the relaxed pins above and every checkpoint passed (logged as BUGLOG B01). Use 3.12 with the exact pins if you have it; otherwise the above is verified-good.
- qiskit-ibm-runtime is **not** in requirements.txt — the notebook itself never needs it. Install it separately, as above.

Save your own credentials once:

```
python -c "from qiskit_ibm_runtime import QiskitRuntimeService as S; \  
S.save_account(channel='ibm_cloud', token='YOUR_TOKEN', overwrite=True)"
```

2. Run it

```
python hardware_run.py --list # confirm the exact backend name  
python hardware_run.py --submit --backend ibm_nighthawk # ~3 s QPU, prints a JOB_ID  
python hardware_run.py --fetch <JOB_ID> # after it completes
```

The run we'd most like from you

```
python hardware_run.py --submit --backend ibm_nighthawk --model 2site --shadow
```

That is the **reduced 2-site instance with a full balanced basis ensemble** — 18 circuits, 36 000 shots, ~30 s of QPU. It is worth more than the default single-point run for two reasons:

1. **It is only 15 two-qubit gates** (depth 64), against 101 for the 3-qubit exact path and 435 for the Trotter circuit that failed on our hardware. On a device with $2e-3$ two-qubit error that predicts ~97% signal survival, i.e. an actually *usable* result.
2. **It can report a valid system observable.** At $n=2$ the whole shadow ensemble is just $3^2=9$ bases, so $\langle Q \rangle$ comes out unbiased. Every fixed-basis job — including all our χ -only jobs — can only legitimately report χ (see below). This is genuinely the one result we have on only ONE device so far (our own 2-site job returned at χ survival 0.962 and a valid $\langle Q \rangle$ at 0.957 — see §3), so it's not "give us something we already have" on a *third* device tier.

If you have QPU time for exactly one thing, run that.

⚠ Fixed basis vs shadow ensemble (BUGLOG B04 — we got this wrong once)

The shadow estimator $3^w \prod_{s,j} 1[b_j = P_j]$ is unbiased **only** for i.i.d.-uniform bases; that random draw is what the 3^w factor compensates for. With a single fixed basis the indicator becomes deterministic and non-matching Pauli terms contribute structurally zero, so the

estimator silently returns a *different observable*. We reported a $\langle Q \rangle$ from fixed-basis hardware data and attributed the discrepancy to noise; on a noiseless simulator the same code is 65σ off, so it was never a noise effect at all.

`hardware_run.py` now refuses to print system observables unless `--shadow` was used. Please don't work around that guard.

That's everything. `hardware_run.py` executes the circuit builder, memory parser and estimators **straight out of the graded notebook**, so it cannot drift from our code — it reimplements no physics. Defaults reproduce our reference point exactly ($t=0.9$, both quadratures, basis $[X,Y,Z]$, 2000 shots, `optimization_level=1`), so your number is directly comparable to ours.

Sanity check before trusting it — these must print survival 0.179 and 0.878 respectively:

```
python hardware_run.py --fetch d9u95vs98n5s7392iao0      # Trotter+marrakesh (the bad one)
python hardware_run.py --fetch d9ujlb0u5hac73ahadu0      # exact+kingston (our best result)
```

2b. The Track B ask (newer, and the one we actually want most)

Everything above is the Part A ask. **If you can only run one thing, run this instead.** Track B (the anti-controlled Hadamard test) has never run on a Nighthawk-class device, and it is where our only genuinely open question sits.

Scripts to hand over

Five files, plus the two they read from. All paths inside them are derived from the repo root, so a clone anywhere works:

file	role
<code>hardware_run.py</code>	required — provides <code>load_notebook_definitions()</code> and <code>get_model()</code>
<code>shadow_hadamard_challenge_PARTICIPANT.ipynb</code>	required — the definitions are exec'd out of it; no kernel needed
<code>evidence/scripts/track_b_hardware_prep.py</code>	go/no-go sizing. Run this FIRST
<code>evidence/scripts/track_b_hardware_submit.py</code>	validates the identity, then submits
<code>evidence/scripts/track_b_hardware_fetch.py</code>	analysis — produces every number we would quote
<code>evidence/scripts/track_b_baselines.py</code>	<i>optional</i> , classical only, no QPU
<code>CONVENTIONS.md</code>	<i>optional</i> — read §5 if any sign looks wrong

Run order

```
python evidence/scripts/track_b_hardware_prep.py  ibm_nighthawk  # 1. sizing, no QPU time
python evidence/scripts/track_b_hardware_submit.py --dry-run      # 2. validation only
python evidence/scripts/track_b_hardware_submit.py  ibm_nighthawk  # 3. submit
python evidence/scripts/track_b_hardware_fetch.py    # 4. when it finishes
```

Budget: 116 circuits × 1000 shots = 116,000 shots, about a minute of QPU time. The job carries four arms so the comparison is same-device, same-hour: our Track-B overlap, our per-observable profile, a Loschmidt echo, and a Hilbert-Schmidt test.

Three things that will bite you, all of which bit us

- **prep.py and submit.py both refuse to proceed if the statevector identity fails.** That is deliberate — leave it in. It is what caught our own endianness bug (we took the `|1>`-control block as the *upper half* of the gate matrix; the notebook builds the ancilla as the

lowest bit, so it is the odd sublattice [1::2,1::2]). The wrong version passes at $t=0$, where both branches are the identity, and fails at every $t>0$.

- **The 2-qubit basis gate is auto-detected from the backend target**, not hardcoded. If Nighthawk uses a gate we have never seen, the scripts will say so and stop rather than silently counting zero 2q gates and reporting survival 1.000. If it *does* stop, send us the printed operation list — do not patch around it.
- **Do not post-select on the charge 0**. It is a diagnostic only; filtering on it biases the shadow estimators (our BUGLOG B04 failure class).

What we predict, so you can falsify it

On `ibm_marrakesh` (2q error $\sim 1.4e-3$) we measured, at $t = 0.0/0.9/1.8/2.7$:

- Loschmidt echo $P_0 = 1.0000 / 0.9650 / 0.5420 / 0.1340$ against exact $1.0000 / 0.9738 / 0.5854 / 0.1069$ — the 2-gate baseline is excellent.
- Our 136-gate Track-B arm: $|\chi_{AB}|$ survival $0.994 / 0.578 / 0.824 / 1.350$. The last point is **above 1**, which is unphysical for pure damping — so the errors there are not simple depolarising loss.
- $\langle Q \rangle_W - \langle Q \rangle_U$ must be **exactly zero** by conservation; we measured $-0.04 / -0.39 / -0.25 / -0.34$.

The interesting question for you: does a better device move our arm toward the echo, or is the gap structural (136 gates vs 2) and therefore permanent? Either answer is publishable for us. If our arm stays bad on good hardware, we will say so on the slide.

2c. The AQC ask (newest, and the one with the biggest headroom)

Section 2b is Track B. **This one is smaller, shorter, and has more to gain from a better device than anything else in the project.**

We claim (R046/R052) that Approximate Quantum Compiling makes a Hadamard test's controlled evolution affordable: at $n=6$ system qubits the standard exact block routes to **8,850** two-qubit gates, controlled Trotter to **2,119**, and controlled AQC to **576**. This job runs all three, as the *same* Hadamard test, and asks whether the gate-count advantage becomes a measured one.

Scripts

file	role
<code>hardware_run.py</code> , <code>shadow_hadamard_challenge_PARTICIPANT.ipynb</code>	required , as in §2b
<code>evidence/scripts/aqc_hardware_submit.py</code>	validates all three arms, then submits
<code>evidence/scripts/aqc_hardware_fetch.py</code>	analysis

Extra dependency beyond §2b: `pip install qiskit-addon-aqc-tensor quimb jax`.

The device name is `ibm_miami` (it *is* the Nighthawk-class machine — see the architecture identification in §2 and R031).

```
# 1. offline sanity check – no backend contact, no cost
python evidence/scripts/aqc_hardware_submit.py --dry-run

# 2. transpile + predictions on miami – still submits NOTHING. Send us this output
#    before spending: it prints the routed gate counts and predicted survivals.
python evidence/scripts/aqc_hardware_submit.py ibm_miami --dry-run

# 3. submit, cheap mode (the deliberately-dead control arm at 500 shots: ~4-5x
#    fewer quantum seconds, no loss to the conclusion)
```

```
python evidence/scripts/aqc_hardware_submit.py ibm_miami --cheap

# 4. when the job shows DONE
python evidence/scripts/aqc_hardware_fetch.py ibm_miami

# 5. ONLY if step 4 shows the AQC arm surviving >= 0.2 (escalation rule below):
python evidence/scripts/aqc_hardware_submit.py ibm_miami --cheap --n 7
python evidence/scripts/aqc_hardware_fetch.py ibm_miami
```

Notes: the record file in the repo contains *our* marrakesh/kingston job ids — your fetch will print "not visible from this account, skipping" for those; that is expected. Each submit stores a self-contained record, so an n=7 run cannot corrupt the n=6 analysis.

Budget: 18 circuits; with --cheap, 4,000 shots on the Trotter/AQC arms and 500 on the exact control $\approx$ 51,000 shots, $\sim$ 6 s of circuit time at n=6.

Our numbers on `ibm_marrakesh`, as falsifiable predictions

arm	2q gates	predicted chi survival
exact block	8850	3.6e-07 (should be noise)
Trotter r=2	2119	1.4e-02
AQC + phase fix	576	0.32

Job d9uv5h50vrcc73boj8a0 was submitted to `ibm_marrakesh` on 2026-08-14; compare against it.

The concrete target: `ibm_miami` (Nighthawk) on the paid/teammate account

The number to beat, from our two Heron runs: the AQC arm's measured survival 0.026–0.033 at 576 routed gates implies an **effective error $\approx 6 \times 10^{-3}$ per two-qubit gate** ($4 \times$ the calibrated gate error — the residual is crosstalk/coherence we could not separate, R054). For the AQC arm to show clear signal it needs **effective error $\leq 2 \times 10^{-3}$** at that depth. Nighthawk is the architecture IBM aims at exactly this regime ($\sim$ 5,000-gate circuits), so `ibm_miami` is the best available shot at flipping our negative into "the compilation win reaches hardware on the newest architecture."

Two things to check before spending money: 1. Your own log records miami's *median calibrated* 2q error as **2.91e-3 — worse than kingston's 2.08e-3**. If miami wins, it wins on coherence/crosstalk, not headline gate error. So run the dry run first and look at the predicted survivals it prints: `python evidence/scripts/aqc_hardware_submit.py ibm_miami --dry-run` 2. Cost: our Heron jobs burned 50–72 quantum seconds each, and **77% of that is the deliberately-dead exact arm**. Submit with `--cheap` (exact arm at 500 shots — still plenty to certify a corpse) and optionally `--shots 2000`; together they cut the quantum-seconds bill roughly 4–5 $\times$ with no loss to the conclusion.

Either outcome is reportable: predictions print before submission, and `aqc_hardware_fetch.py` compares against them. If miami's AQC arm also returns noise, that is a three-architecture negative and we say so.

Flags (all verified by dry run): `--n 7` runs the larger instance (default 6 — and we recommend keeping 6: at n=7 the exact control arm alone is 36,114 routed gates, $4.1 \times$ the quantum time, while the AQC arm's expected signal *halves*; the full arithmetic is R055 and appendix A7 of the Track B deck). `--shots N` and `--cheap` as above.

Escalation rule — run n=7 only as a follow-up, never first: - Run `--cheap` at the default n=6 and fetch. A null at n=7-first is uninterpretable (device not good enough vs n too deep); n=6 separates those. - **If** the n=6 AQC arm survives $\geq \sim 0.2$ ($\Rightarrow$ effective error $\leq 2.8e-3$ /gate), escalate: `--n 7 --cheap`. At that error rate the 702-gate n=7 arm predicts 0.15–0.25 — measurable, and it answers a new question (does the win *scale* on hardware). With `--cheap` the n=7 job is only $\sim 2 \times$ the n=6 circuit time. - If n=6 fails on miami too, stop: that is a clean three-architecture negative at matched n, and n=7 would add cost, not information. Note `--dry-run`

with no `ibm_*` argument is statevector-only and touches no backend; with a backend name it also transpiles and prints the cost table without submitting.

Why a better device matters more here than anywhere else

Our predicted AQC survival is only ~ 0.32 on marrakesh -- good enough to see a signal, not good enough to be comfortable. On a device with lower two-qubit error the AQC arm should climb sharply while the exact arm stays dead (it is 8,850 gates; nothing rescues that). **The cleaner the device, the more decisive this experiment gets** -- which is the opposite of most of our runs, where a better device would only have made an already-fine number slightly finer.

Three things that will bite you

- **Do not remove the $P(-\theta)$ on the ancilla.** AQC-Tensor optimises state fidelity, which is blind to global phase; a Hadamard test measures precisely that phase. Without the correction chi comes back wrong by ~ 3 radians and the run is worthless. The script computes theta at compile time and applies it; the pre-flight check will fail loudly if it is dropped (R046).
- **The script refuses to submit if the statevector pre-flight fails.** Leave that in.
- **The exact arm is expected to fail.** That is the point of including it, not a bug -- do not "fix" it by lowering n or raising shots.

2d. The fix-A/B ask (added 2026-08-16 — the cheapest and the one the paper wants most)

What it is. On 2026-08-16 we demonstrated the phase trap directly on `ibm_marrakesh` (R061): the AQC-compiled Hadamard test WITH and WITHOUT the one-gate phase fix, in one job at $n=3$ — shallow enough that signal survives. Result: the two arms are indistinguishable to every magnitude metric (pre-registered P1), while the compile-time phase offset between them survives ~ 186 gates of device noise to within 0.06-0.13 rad of prediction (P2). Instructively, the device also adds its own coherent phase error (P3 falsified) — even the exact reference arm, at survival 0.86, carries a 0.26-0.44 rad phase error its survival number cannot see.

The ask: repeat this exact job on `ibm_miami`. One device, one architecture is the paper's stated limitation. A square-lattice (Nighthawk) repetition answers two questions our account cannot: is P1/P2 architecture-independent, and is the device-induced coherent phase (the P3 surprise) universal or heavy-hex-specific?

```
# 1. offline pre-flight – compiles the AQC circuits, verifies all three arms on the
#   statevector, prints the pre-registered predictions. No backend contact.
python evidence/scripts/fix_ab_hardware.py --dry-run

# 2. submit (18 circuits x 4000 shots; these are n=3 circuits, ~100-190 routed 2q
#   gates – the whole job is a fraction of one AQC-ask job's cost)
python evidence/scripts/fix_ab_hardware.py ibm_miami

# 3. when DONE
python evidence/scripts/fix_ab_hardware.py --fetch
```

Notes: the record file `evidence/fix_ab_job.json` already contains our marrakesh job id; your fetch will print "not visible from this account, skipping" for it — expected. Your submission merges under its own backend key and cannot clobber ours. Extra dependency beyond §2b: `pip install qiskit-addon-aqc-tensor quimb jax` (same as §2c).

What to look at in the fetch output, against our marrakesh numbers (R061): the "P2 check" lines — measured `arg(naive/fixed)` vs the pre-registered theta printed at submission (ours deviated 0.056/0.058/0.133 rad at $t=0.3/0.6/0.9$); and each arm's "phase err vs exact" — ours showed the deep arms carrying a common ~ -2.2 rad device-induced phase the fix cannot anticipate. If miami's device phase is small, that is a genuinely new architecture finding; if it is

large too, the paper's "phase errors demand phase-sensitive certification" claim gets a second architecture. Either outcome is reportable.

3. What we already know (please don't re-derive this)

Updated 2026-08-13 — all four arms of the 2×2 have now returned, plus both side-model jobs. The result is good news.

The single most important finding: **circuit depth dominates device quality, by a lot** — and once you fix the depth, real hardware works.

circuit	two-qubit gates (kingston)	depth	measured survival
Trotter reps=1, marrakesh (worst device)	435	1729	0.179 (35σ off — effectively no signal)
Trotter reps=1, kingston	435	1729	0.368
exact path, marrakesh	101	375	0.822 (5.8σ, 5.0σ)
exact path, kingston	101	375	0.878 (4.4σ, 0.9σ) — best result we have
2-site side model, kingston (full shadow ensemble)	15	64	0.962 , and a valid ⟨Q⟩ at 0.957

So: changing only the circuit (Trotter → exact, same device) took survival from 0.18 to 0.82 — roughly 4.6×. Changing only the device on top of that (marrakesh → kingston, exact path both times) added another ~7% (0.822 → 0.878). Depth dominates; device choice is a smaller, real, additional effect. (One caveat we're keeping honest: this is a single-run comparison per arm, not a repeated-trial average — treat the *pattern*, depth >> device, as solid; treat the exact percentages as one data point each.)

The depth effect is now confirmed on **both** devices: exact/Trotter is 4.59× on marrakesh and 2.39× on kingston. Device choice at fixed circuit is smaller (1.07× exact, 2.06× Trotter). Depth dominates throughout.

So please use `--method exact` (the default) — it's not just theoretically better, it's now confirmed to work on two real devices. `--method trotter` is available if you want the deep-circuit arm for comparison on Nighthawk, but expect it to be mostly noise based on what we've seen.

Kingston (2q error 1.99e-3) has been our best-calibrated and best-performing device so far. If Nighthawk's error rates are in the same range or better, we'd expect survival in the 0.85-0.95+ region on the exact path — genuinely comparable to a good simulator result.

4. What to send back

Just the console output of `--fetch`. The line that matters is:

```
SIGNAL SURVIVAL |chi_hw|/|chi_exact| = 0.XXX
```

Plus the backend name and its median 2q error from `--list`, so we can place it on the depth-vs-fidelity curve. If you want to be maximally helpful, run **both** `--method exact` and `--method trotter` — that gives us the depth axis on your device too.

5. Rules we're holding ourselves to

Please keep these if you report anything publicly:

- **Robustness claims only.** No advantage claims. The system is 3 qubits and a laptop diagonalises it instantly — it is small *on purpose* so every estimate can be graded against an exact answer.
- **The exact path is not a method recommendation.** It is shallower only because $n=3$ is tiny (exact synthesis costs $O(4^n)$); the advantage inverts as n grows. It is legitimate as a robustness demonstration, never as an algorithmic claim.
- **Every number traces to a command.** If you report a figure, say which invocation produced it — that's the rule the whole repo runs on (see RESULTS.md).

6. Background, if you want it

- EXPLAINER.md — plain-language description of what the project does
- RESULTS.md — every number, with the command that produced it (R008 and R018 are hardware)
- BUGLOG.md — B03 is the initial hardware finding, B04 is a real mistake we made and fixed (reported a hardware $\langle Q \rangle$ from a fixed-basis job — wrong, since the shadow estimator needs random bases; caught it, retracted it, added the `--shadow` guard in this tool)
- CONVENTIONS.md — the frozen conventions; §2 has the Hamiltonian, §5 the shot budget
- figures/07_symmetry_resolved_spectrum.png — the headline result the hardware run is ultimately trying to support